```
34
35      if(fx != fy) return -1;
36      else return abs(d[y] - d[x]) - 1;
37  }
38
39  int main()
40  {
41      // 初始化
42      for(int i = 1; i <= n; i++)
43      {
44          fa[i] = i;
45          cnt[i] = 1;
46      }
47
48      int T; cin >> T;
49
50      while(T--)
51      {
52          char op; int x, y;
53          cin >> op >> x >> y;
54          if(op == 'M') // 合并
55          {
56              un(x, y);
57          }
58          else // 查询
59          {
60              cout << query(x, y) << endl;
61          }
62      }
63
64      return 0;
65  }
```

4. 字符串哈希

1. 回忆：哈希函数与哈希冲突

- **哈希函数：**将关键字映射成对应的地址的函数，记为 `Hash(key) = Addr`。
- **哈希冲突：**哈希函数可能会把两个或两个以上的不同关键字映射到同一地址，这种情况称为哈希冲突。

2. 字符串哈希

定义一个把字符串映射到整数的函数 $hash$，这就是字符串哈希。说白了，就是将一个字符串用一个整数表示。

3. 字符串哈希中的哈希函数

在字符串哈希中，有一种冲突概率较小的哈希函数，将字符串映射成 p 进制数字：

$$hash(s) = \sum_{i=0}^{n-1} s[i] \times p^{n-i-1} \ (\mod M)$$

其中，p 通常取质数 131 或者 13331。如果把哈希值定义为 `unsigned long long` 类型，在 C++ 中，溢出就会自动取模。

（你没有看错，字符串哈希就是背一个公式即可……）

但是，实际求哈希值时，我们用的是前缀哈希的思想来求，这样会和下面的多次询问子串哈希一致。

4. 前缀哈希数组

单次计算一个字符串的哈希值复杂度是 $O(N)$。如果需要多次询问一个字符串的子串的哈希值，每次重新计算效率非常低下。

一般利用前缀和思想先预处理字符串中每个前缀的哈希值，这样的话每次就能快速求出子串的哈希了。

```
typedef unsigned long long ULL;
const int N = 1e6 + 10, P = 13331;
char s[N];
int len;
ULL f[N]; // 前缀哈希数组
ULL p[N]; // 记录 p 的 i 次方

// 处理前缀哈希数组以及 p 的 i 次方数组
void init_hash()
{
    f[0] = 0; p[0] = 1;
    for(int i = 1; i <= len; i++)
    {
        f[i] = f[i - 1] * P + s[i];
        p[i] = p[i - 1] * P;
    }
}

// 快速求得任意区间的哈希值
ULL get_hash(int l, int r)
{
```

```
    return f[r] - f[l - 1] * p[r - l + 1];
}
```

如果题目只是简单的求单个字符串的哈希值：

```
typedef unsigned long long ULL;
const int N = 1e6 + 10;
int len;
char s[N];

ULL gethash()
{
    ULL ret = 0;
    for(int i = 1; i <= len; i++)
    {
        ret = ret * p + s[i];
    }
    return ret;
}
```

4.1 【模板】字符串哈希

> **题目来源：** 洛谷
>
> **题目链接：** P3370 【模板】字符串哈希
>
> **难度系数：** ★★

【题目描述】

> 如题，给定 N 个字符串（第 i 个字符串长度为 M_i，字符串内包含数字、大小写字母，大小写敏感），请求出 N 个字符串中共有多少个不同的字符串。
>
> **友情提醒：如果真的想好好练习哈希的话，请自觉。**

【输入描述】

> 第一行包含一个整数 N，为字符串的个数。
>
> 接下来 N 行每行包含一个字符串，为所提供的字符串。

【输出描述】

> 输出包含一行，包含一个整数，为不同的字符串个数。

【示例一】

> 输入：
>
> 5
>
> abc
>
> aaaa
>
> abc
>
> abcc
>
> 12345
>
> 输出：
>
> 4

【解法】

字符串哈希模板题，就是背一个哈希函数~

【参考代码】

```cpp
#include <iostream>
#include <algorithm>

using namespace std;

typedef unsigned long long ULL;

const int N = 1e4 + 10, P = 131;

int n;
ULL a[N];

// 字符串哈希
ULL get_hash(string& s)
{
    ULL ret = 0;
    for(int i = 1; i <= s.size(); i++)
    {
        ret = ret * P + s[i - 1];
    }

    return ret;
}

int main()
{
```

```
27        cin >> n;
28        for(int i = 1; i <= n; i++)
29        {
30            string s; cin >> s;
31            a[i] = get_hash(s);
32        }
33
34        int ret = 1;
35        sort(a + 1, a + 1 + n);
36        for(int i = 2; i <= n; i++)
37        {
38            if(a[i] != a[i - 1]) ret++;
39        }
40
41        cout << ret << endl;
42
43        return 0;
44    }
```

4.2 兔子与兔子

题目来源： 洛谷

题目链接： P10468 兔子与兔子

难度系数： ★★

【题目描述】

很久很久以前，森林里住着一群兔子。

有一天，兔子们想要研究自己的 DNA 序列。

我们首先选取一个好长好长的 DNA 序列（小兔子是外星生物，DNA 序列可能包含 26 个小写英文字母）。

然后我们每次选择两个区间，询问如果用两个区间里的 DNA 序列分别生产出来两只兔子，这两个兔子是否一模一样。

注意两个兔子一模一样只可能是他们的 DNA 序列一模一样。

【输入描述】

第一行输入一个 DNA 字符串 S。

第二行一个数字 m，表示 m 次询问。

接下来 m 行，每行四个数字 l1,r1,l2,r2，分别表示此次询问的两个区间，注意字符串的位置从 1 开始编号。

【输出描述】

> 对于每次询问，输出一行表示结果。
>
> 如果两只兔子完全相同输出 `Yes`，否则输出 `No`（注意大小写）。

【示例一】

> 输入：
>
> aabbaabb
>
> 3
>
> 1 3 5 7
>
> 1 3 6 8
>
> 1 2 1 2
>
> 输出：
>
> Yes
>
> No
>
> Yes

【解法】

构造字符串的前缀哈希数组，在前缀哈希数组中快速拿到区间的哈希值。

【参考代码】

```cpp
#include <iostream>

using namespace std;

typedef unsigned long long ULL;

const int N = 1e6 + 10, P = 13331;

int n;
string s;
ULL f[N]; // 前缀哈希数组
ULL p[N]; // p 的 i 次方

void init_hash()
{
    p[0] = 1;

    for(int i = 1; i <= n; i++)
```

```
    {
        f[i] = f[i - 1] * P + s[i];
        p[i] = p[i - 1] * P;
    }
}

ULL get_hash(int l, int r)
{
    return f[r] - f[l - 1] * p[r - l + 1];
}

int main()
{
    cin >> s;
    n = s.size();
    s = " " + s;

    init_hash();

    int m; cin >> m;
    while(m--)
    {
        int l1, r1, l2, r2; cin >> l1 >> r1 >> l2 >> r2;

        ULL x = get_hash(l1, r1), y = get_hash(l2, r2);

        if(x == y) cout << "Yes" << endl;
        else cout << "No" << endl;
    }

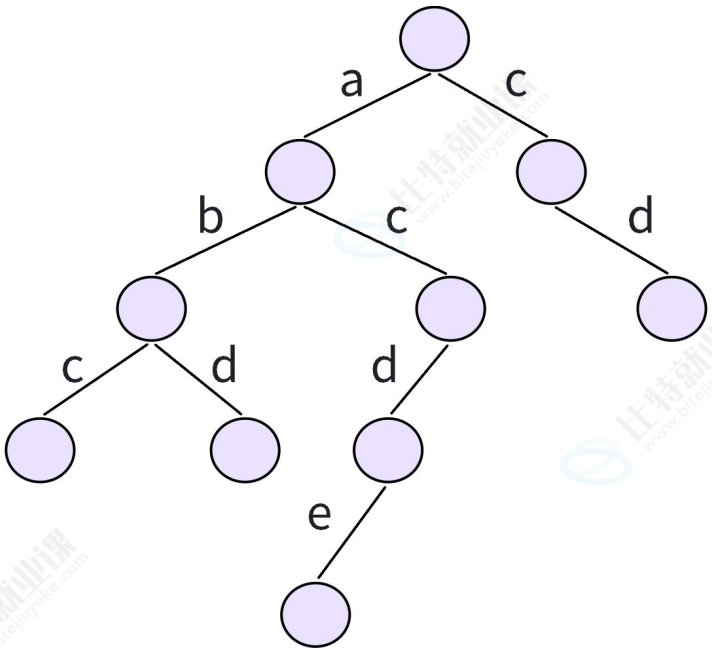
    return 0;
}
```

5. Trie 树

1. 字典树的概念

Trie 树又叫字典树或前缀树，是一种能够快速插入和查询字符串的数据结构。它利用字符串的公共前缀，将字符串组织成一棵树形结构，从而大大提高了存储以及查找效率。

我们可以把字典树想象成一棵多叉树，每一条边代表一个字符，从根节点到某个节点的路径就代表了一个字符串。例如，要存储 `"abc"`、`"abd"`、`"acde"` 以及 `"cd"` 时，构建的字典树如下：



2. 字典树的作用

当我们在字典树的每一个结点位置，额外维护一些信息时，就可以做到很多事情：

- 查询某个单词是否出现过，并且出现几次；
- 查询有多少个单词是以某个字符串为前缀；
- 查询所有以某个前缀开头的单词；（这个作用可以用到输入法中，输入拼音的时候，可以提示可能的单词）

当然，除了上述作用以外，字典树还可以解决别的问题，后续可以在做题中体会。

3. 字典树的实现

实现一个能够**查询单词出现次数**以及**查询有多少个单词是以某个字符串为前缀**的字典树，默认全是小写字母。

准备工作：

```cpp
#include <iostream>
#include <cstring>

using namespace std;
const int N = 1e6 + 10;

int tree[N][26], p[N], e[N];
```

```
int idx;
```

插入字符串：

```
void insert(string& s)
{
    int cur = 0; // 从根结点开始
    p[cur]++; // 这个格子经过一次

    for(auto ch : s)
    {
        int path = ch - 'a';
        // 如果没有路
        if(tree[cur][path] == 0) tree[cur][path] = ++idx;
        cur = tree[cur][path];
        p[cur]++;
    }
    e[cur]++;
}
```

查询字符串出现的次数：

```
int find(string& s)
{
    int cur = 0;
    for(auto ch : s)
    {
        int path = ch - 'a';
        if(tree[cur][path] == 0) return 0;
        cur = tree[cur][path];
    }
    return e[cur];
}
```

查询有多少个单词以字符串 `s` 为前缀：

```
int find_pre(string& s)
```

```
    {
        int cur = 0;
        for(auto ch : s)
        {
            int path = get_num(ch);
            if(tree[cur][path] == 0) return 0;
            cur = tree[cur][path];
        }
        return p[cur];
    }
```

5.1 【模板】字典树

> **题目来源：** 洛谷
>
> **题目链接：** P8306 【模板】字典树
>
> **难度系数：** ★★

【题目描述】

> 给定 n 个模式串 $s_1, s_2, \ldots, s_n$ 和 q 次询问，每次询问给定一个文本串 t_i，请回答 $s_1 \sim s_n$ 中有多少个字符串 s_j 满足 t_i 是 s_j 的前缀。
>
> 一个字符串 t 是 s 的前缀当且仅当从 s 的末尾删去若干个（可以为 0 个）连续的字符后与 t 相同。
>
> 输入的字符串大小敏感。例如，字符串 `Fusu` 和字符串 `fusu` 不同。

【输入描述】

> **本题单测试点内有多组测试数据。**
>
> 输入的第一行是一个整数，表示数据组数 T。
>
> 对于每组数据，格式如下：
> 第一行是两个整数，分别表示模式串的个数 n 和询问的个数 q。
> 接下来 n 行，每行一个字符串，表示一个模式串。
> 接下来 q 行，每行一个字符串，表示一次询问。

【输出描述】

> 按照输入的顺序依次输出各测试数据的答案。
>
> 对于每次询问，输出一行一个整数表示答案。

【示例一】

> 输入：
>
> ```
> 3
> ```

3 3

fusufusu

fusu

anguei

fusu

anguei

kkksc

5 2

fusu

Fusu

AFakeFusu

afakefusu

fusuisnotfake

Fusu

fusu

1 1

998244353

9

输出：

2

1

0

1

2

1

【解法】

模板题，仅需维护一个 p 数组，查询多少前缀即可。

【参考代码】

```
1   #include <iostream>
```

```cpp
#include <cstring>

using namespace std;

const int N = 3e6 + 10;

int n, q;
int tr[N][62], p[N];
int idx;

int get_num(char ch)
{
    if(ch >= 'a' && ch <= 'z') return ch - 'a';
    else if(ch >= 'A' && ch <= 'Z') return ch - 'A' + 26;
    else return ch - '0' + 52;
}

void insert(string& s)
{
    int cur = 0;
    p[cur]++;

    for(auto ch : s)
    {
        int path = get_num(ch);
        if(tr[cur][path] == 0) tr[cur][path] = ++idx;

        cur = tr[cur][path];
        p[cur]++;
    }
}

int find_pre(string& s)
{
    int cur = 0;

    for(auto ch : s)
    {
        int path = get_num(ch);
        if(tr[cur][path] == 0) return 0;

        cur = tr[cur][path];
    }
    return p[cur];
}

int main()
```

```cpp
{
    int T; cin >> T;

    while(T--)
    {
        // 清空之前的数据
        // memset(tr, 0, sizeof tr);
        // memset(p, 0, sizeof p);
        for(int i = 0; i <= idx; i++)
        {
            for(int j = 0; j < 62; j++)
            {
                tr[i][j] = 0;
            }
        }

        for(int i = 0; i <= idx; i++) p[i] = 0;

        idx = 0;

        cin >> n >> q;
        while(n--)
        {
            string s; cin >> s;
            insert(s);
        }

        while(q--)
        {
            string s; cin >> s;
            cout << find_pre(s) << endl;
        }
    }


    return 0;
}
```

5.2 于是他错误的点名开始了

> **题目来源：** 洛谷
>
> **题目链接：** P2580 于是他错误的点名开始了
>
> **难度系数：** ★★

【题目描述】

> 这之后校长任命你为特派探员，每天记录他的点名。校长会提供化学竞赛学生的人数和名单，而你需要告诉校长他有没有点错名。（为什么不直接不让他玩炉石。）

【输入描述】

> 第一行一个整数 n ，表示班上人数。
>
> 接下来 n 行，每行一个字符串表示其名字（互不相同，且只含小写字母，长度不超过 50 ）。
>
> 第 $n+2$ 行一个整数 m ，表示教练报的名字个数。
>
> 接下来 m 行，每行一个字符串表示教练报的名字（只含小写字母，且长度不超过 50 ）。

【输出描述】

> 对于每个教练报的名字，输出一行。
>
> 如果该名字正确且是第一次出现，输出 `OK` ，如果该名字错误，输出 `WRONG` ，如果该名字正确但不是第一次出现，输出 `REPEAT` 。

【示例一】

> 输入：
>
> ```
> 5
> a
> b
> c
> ad
> acd
> 3
> a
> a
> e
> ```
>
> 输出：
>
> ```
> OK
> REPEAT
> WRONG
> ```

【解法】

用字典树维护字符串，当查询之后，将 e 数组修改成 -1 即可，

【参考代码】

```cpp
#include <iostream>

using namespace std;

const int N = 5e5 + 10;

int n, m;
int tr[N][26], e[N], idx;

void insert(string& s)
{
    int cur = 0;

    for(auto ch : s)
    {
        int path = ch - 'a';
        if(tr[cur][path] == 0) tr[cur][path] = ++idx;
        cur = tr[cur][path];
    }
    e[cur]++;
}

int find(string& s)
{
    int cur = 0;

    for(auto ch : s)
    {
        int path = ch - 'a';
        if(tr[cur][path] == 0) return 0;
        cur = tr[cur][path];
    }
    // e[cur]

    if(e[cur] > 0)
    {
        int t = e[cur];
        e[cur] = -1;
        return t;
    }

    return e[cur];
}

```

```cpp
int main()
{
    cin >> n;
    while(n--)
    {
        string s; cin >> s;
        insert(s);
    }

    cin >> m;
    while(m--)
    {
        string s; cin >> s;
        int ret = find(s);
        if(ret == 0) cout << "WRONG" << endl;
        else if(ret > 0) cout << "OK" << endl;
        else cout << "REPEAT" << endl;
    }

    return 0;
}
```

5.3 最大异或对

> **题目来源：** 洛谷
>
> **题目链接：** P10471 最大异或对 The XOR Largest Pair
>
> **难度系数：** ★★★

【题目描述】

> 给定 N 个整数 $A_1, A_2, \cdots, A_N$ 中选出两个进行异或计算，得到的结果最大是多少？

【输入描述】

> 第一行一个整数 N，第二行 N 个整数 $A_1, A_2, \cdots, A_N$。

【输出描述】

> 一个整数表示答案。

【示例一】

> 输入：
>
> ```
> 3
> 1 2 3
> ```

> 输出：
>
> 3

【解法】

将所有数按照二进制位存在字典树中，针对每一个数 x ，按照二进制匹配最优的方案。

【参考代码】

```
#include <iostream>

using namespace std;

const int N = 1e5 + 10;

int n;
int a[N];

int tr[N * 32][2], idx;

void insert(int x)
{
    int cur = 0;
    for(int i = 31; i >= 0; i--)
    {
        int path = ((x >> i) & 1);
        if(tr[cur][path] == 0) tr[cur][path] = ++idx;
        cur = tr[cur][path];
    }
}

int find(int x)
{
    int cur = 0;
    int ret = 0;
    for(int i = 31; i >= 0; i--)
    {
        int path = ((x >> i) & 1);

        // 贪心：要去相反的路
        if(tr[cur][path ^ 1])
        {
            ret |= (1 << i);
            cur = tr[cur][path ^ 1];
        }
```

```
37          else
38          {
39              cur = tr[cur][path];
40          }
41      }
42
43      return ret;
44  }
45
46  int main()
47  {
48      cin >> n;
49      for(int i = 1; i <= n; i++)
50      {
51          cin >> a[i];
52          insert(a[i]);
53      }
54
55      int ret = 0;
56      for(int i = 1; i <= n; i++)
57      {
58          ret = max(ret, find(a[i]));
59      }
60
61      cout << ret << endl;
62
63      return 0;
64  }
```